

CSC1031:OOP

Misha

Design Pattern

Decorator



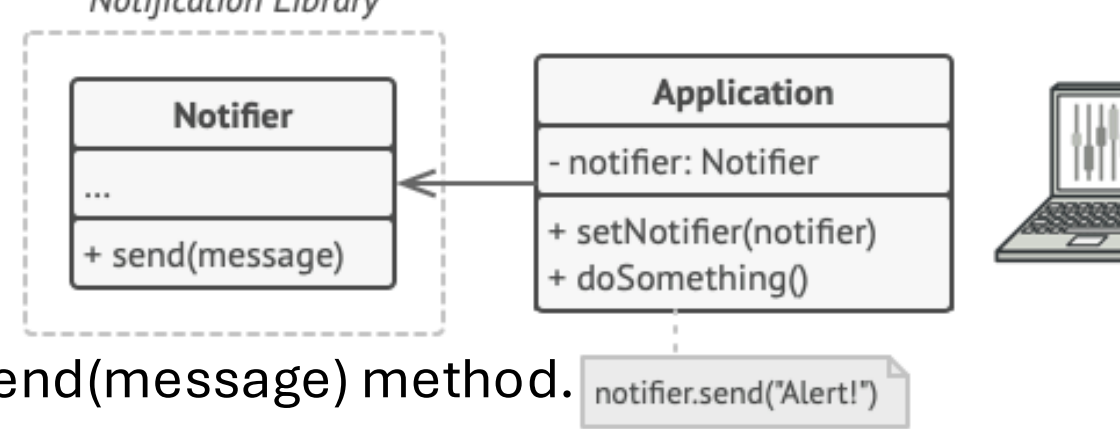
Definition

- Decorator is a **structural design** pattern that lets you attach new behaviors to objects by placing these objects inside special wrapper objects that contain the behaviors.



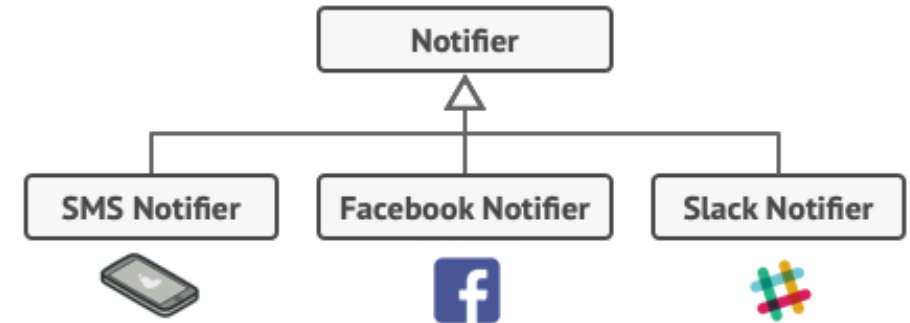
Problem

- The Notifier class was the core of the library.
- It had a constructor for setting a list of emails and a send(message) method.
- Third-party clients created and reused a Notifier instance to send email alerts.



Limitations

- Clients began requesting support for other channels:
 - SMS
 - Facebook
 - Slack

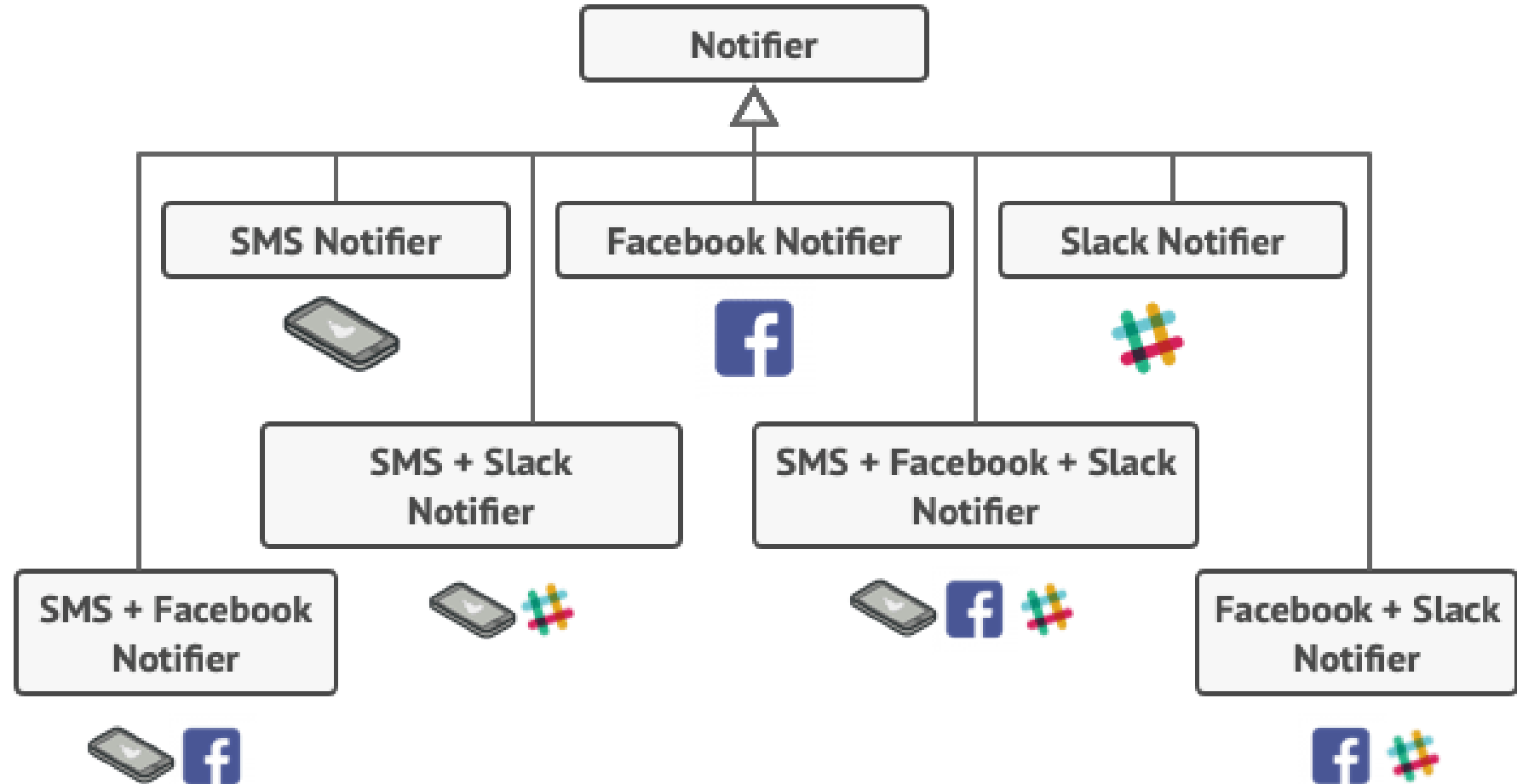


First Attempt at Extending Functionality

- Subclassed Notifier to implement each new notification type.
- Clients used a specific subclass for their preferred channel.

Problem

- Users demanded **multi-channel** notifications.
- Single-inheritance model failed to scale for multiple combinations.



The Pitfall of Subclass Combinations

Attempted Solution: Subclass Combinations

- Created composite subclasses to support multiple channels (e.g., EmailAndSMSNotifier, SlackAndFacebookNotifier, etc.)

Result: Combinatorial Explosion

- Number of subclasses grew exponentially with each added channel.
- Increased **complexity**, **redundancy**, and **maintenance overhead**.

Impact

- Library became inflexible and hard to extend.
- Client code was burdened with selecting and configuring the correct composite class.

Inheritance

- Extending a class is the first thing that comes to mind when you need to alter an object's behavior.
- However, inheritance has several serious caveats that you need to be aware of:
 - **Inheritance is static.**
 - You can't alter the behavior of an existing object at runtime. You can only replace the whole object with another one that's created from a different subclass.
 - **Subclasses can have just one parent class.**
 - In most languages, inheritance doesn't let a class inherit behaviors of multiple classes at the same time.

Solution

- One of the ways to overcome these caveats is by using **Aggregation** or **Composition** instead of Inheritance.



- *Aggregation*: object A contains objects B; B can live without A.
- *Composition*: object A consists of objects B; A manages life cycle of B; B can't live without A.

Composition and Aggregation as Alternatives

Core Idea

- Replace inheritance with **composition** or **aggregation**.

How It Works

- An object holds references to other objects ("helpers") and delegates work.
- Behavior can be modified by replacing the referenced object(s) at runtime.

Benefits

- **Flexible**: Swap components dynamically.
- **Modular**: Combine multiple behaviors easily.
- **Reusable**: Avoids duplication across subclasses.

Design Principle

- Favor composition over inheritance.

Decorator Pattern

Concept

- Also known as a **wrapper** pattern.
- A decorator implements the same interface as the wrapped object.
- It delegates calls to the wrapped object but may extend or modify behavior.

Client Perspective

- Decorated and base objects are **interchangeable**.
- Decorators can be **stacked** for cumulative behavior.

Key Advantage

- Enables dynamic composition of responsibilities.

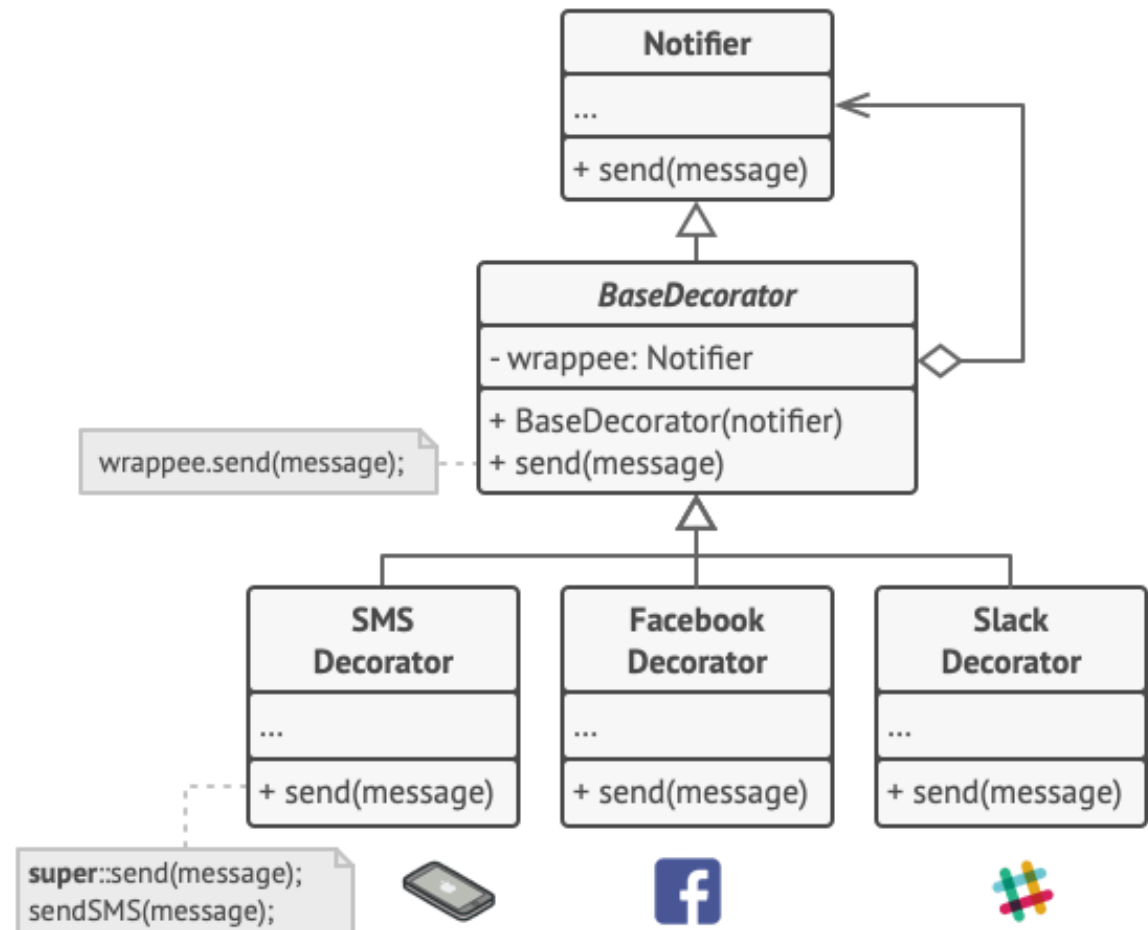
Applying the Decorator Pattern to Notifications

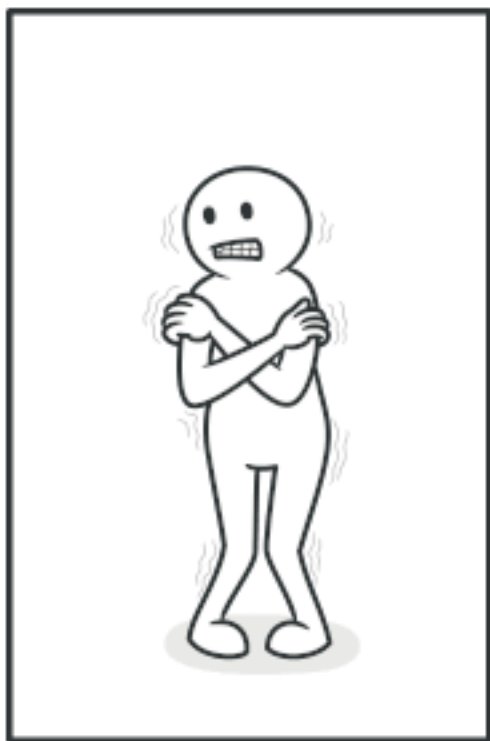
Structure

- Notifier: Base class with email notifications.
- Other channels (SMS, Slack, etc.) implemented as decorators.

Usage

- Clients build a **stack of decorators** based on user preferences.
- The final decorator in the stack is used as the notifier.



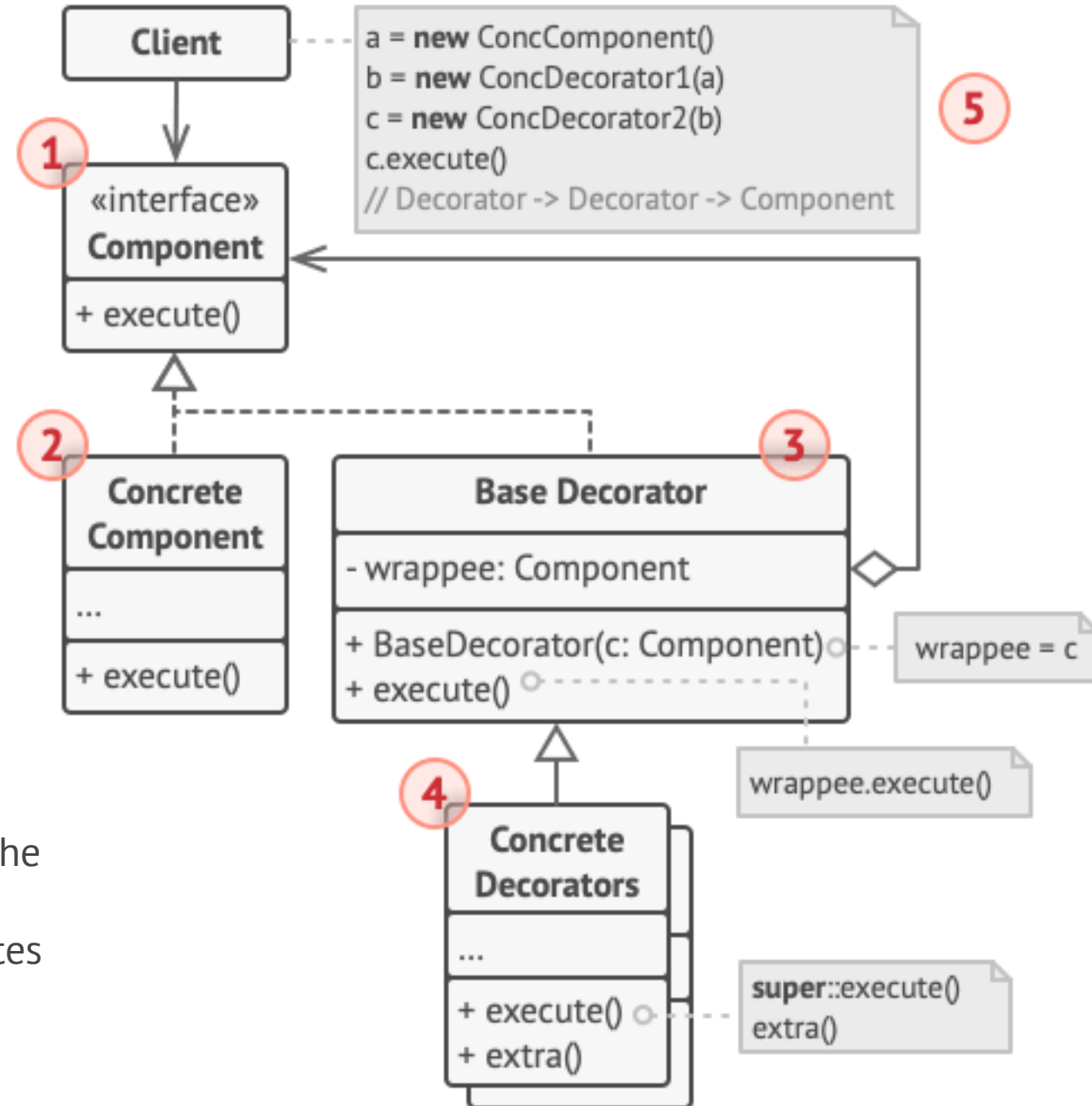


Structure

1. The **Component** declares the common interface for both wrappers and wrapped objects.

2. **Concrete Component** is a class of objects being wrapped. It defines the basic behavior, which can be altered by decorators.

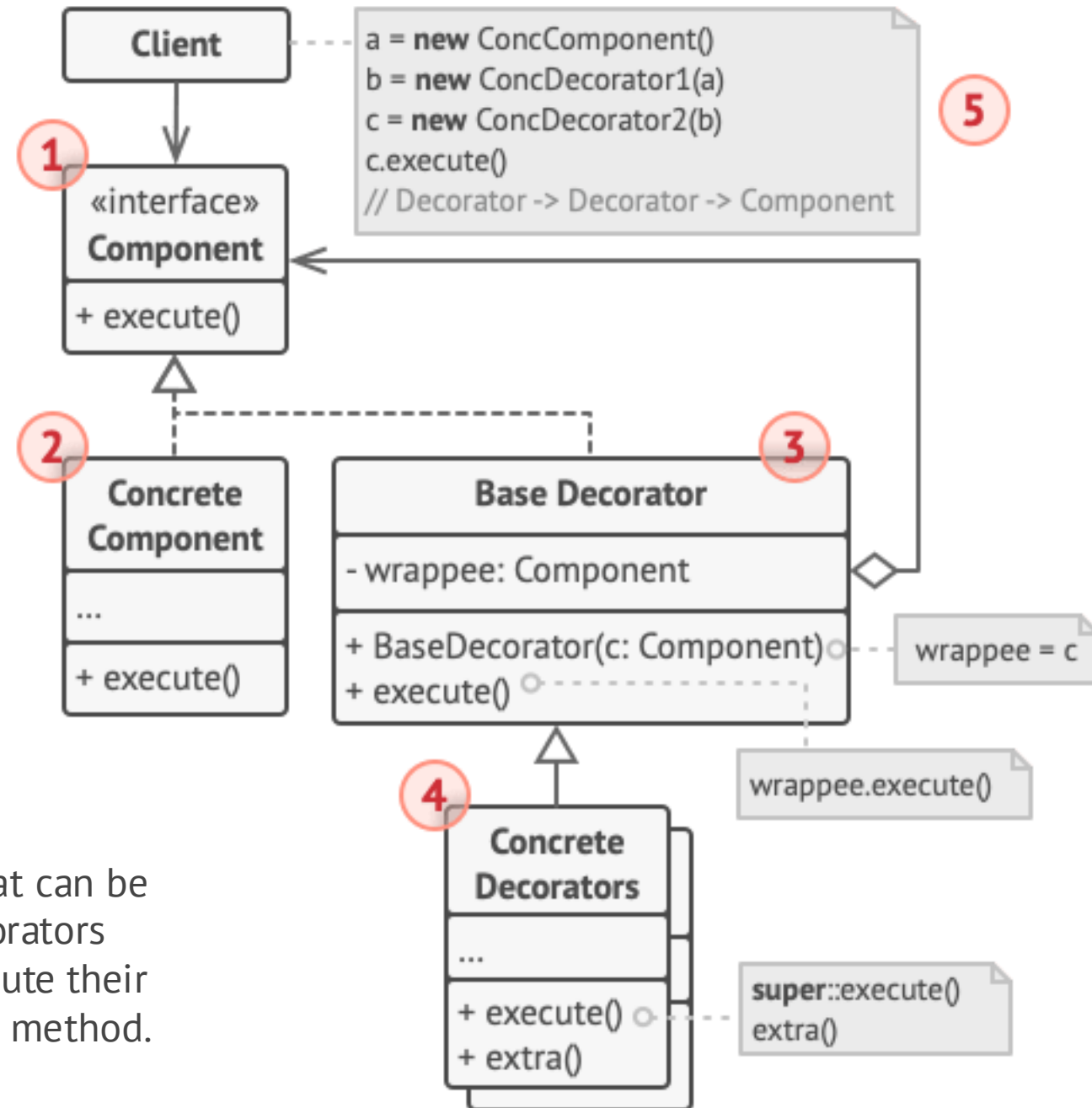
3. The **Base Decorator** class has a field for referencing a wrapped object. The field's type should be declared as the component interface so it can contain both concrete components and decorators. The base decorator delegates all operations to the wrapped object.



Structure

5. The **Client** can wrap components in multiple layers of decorators, as long as it works with all objects via the component interface.

4. **Concrete Decorators** define extra behaviors that can be added to components dynamically. Concrete decorators override methods of the base decorator and execute their behavior either before or after calling the parent method.



When to use?

- Use the Decorator pattern when you need to be able to **assign extra behaviors to objects** at runtime without breaking the code that uses these objects.
- Use the pattern when it's **awkward or not possible** to extend an object's behavior using **inheritance**.

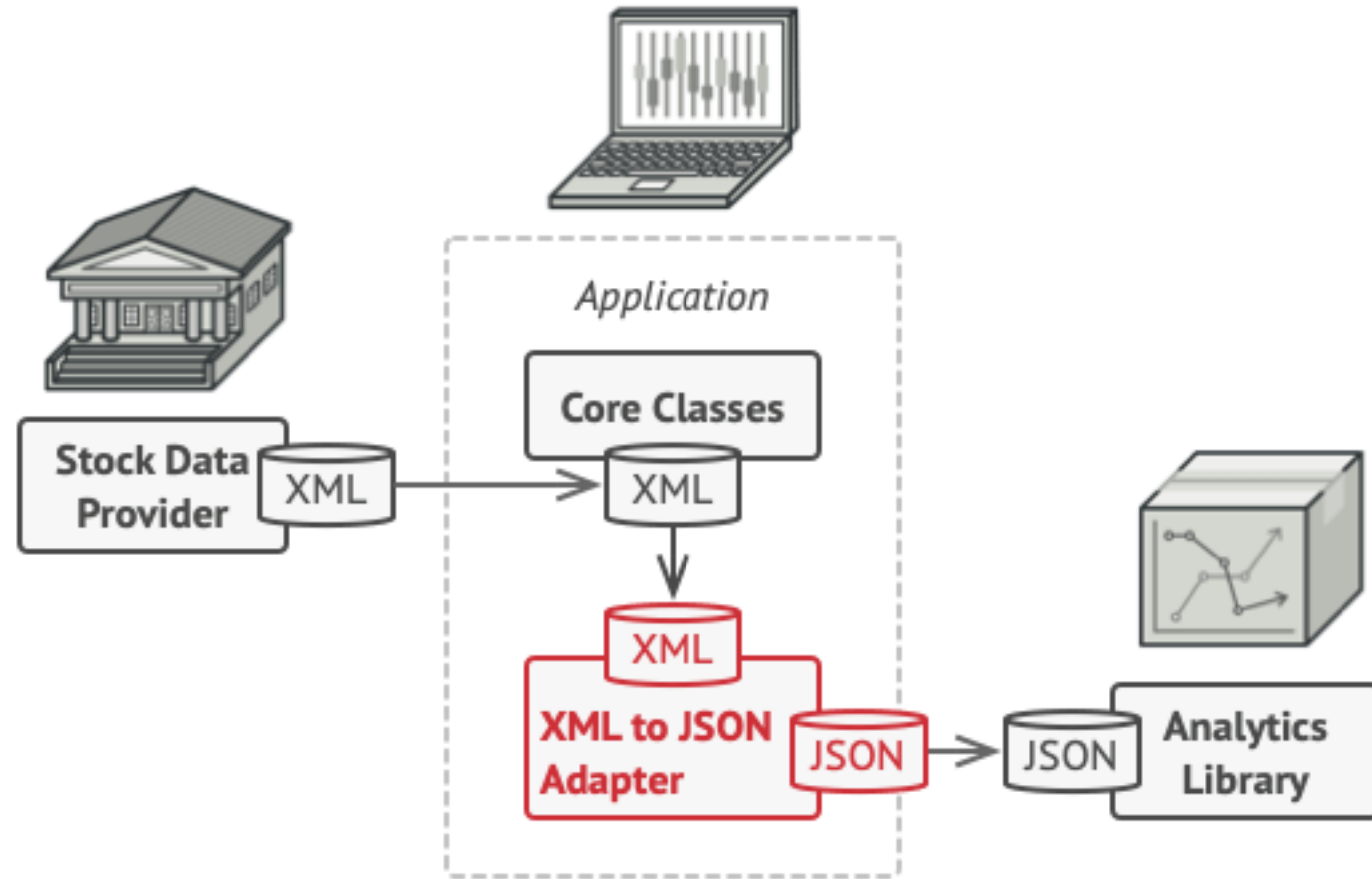
Pros and Cons

- You can extend an object's behavior without making a new subclass.
- You can add or remove responsibilities from an object at runtime.
- You can combine several behaviors by wrapping an object into multiple decorators.
- *Single Responsibility Principle.*
 - You can divide a monolithic class that implements many possible variants of behavior into several smaller classes.

Pros and Cons

- It's hard to remove a specific wrapper from the wrappers stack.
- It's hard to implement a decorator in such a way that its behavior doesn't depend on the order in the decorators stack.
- The initial configuration code of layers might look pretty ugly.

Adapter



Decorator vs Adapter

- **Adapter:** Translates interface.
 - Allows incompatible interfaces to work together
- **Decorator:** Preserves or extends interface, adds behavior recursively.
 - Adds additional responsibilities dynamically